PROGREE

Python Programming Internship

Technical Report — Tasks 2, 3 and 4

Report date	13 October 2026
Evidence source	Fresh Phase 5 validation from commit `0ead4fcfc3a9`
Scope	Fibonacci generation, safe file automation, and a rule-based terminal chatbot
Identity	No participant identity, registration number, or organizer endorsement is represented.

Prepared from executed local evidence only. This report does not confirm certificate eligibility, form submission, or an organizer deadline hour/timezone.

Executive summary. Assignment objectives and verification

The authoritative Python assignment asks for a Fibonacci module with timeit benchmarking, a safe file-organization and text-parsing utility, and a multi-intent terminal chatbot. This report covers the three completed technical tasks and the fresh validation run used for this document.

• The consolidated suite executed 54 tests and passed in 1.678 seconds.

• Task 2 evidence contains real JSON outputs, an invalid-input diagnostic, and 5-repeat × 100-call timeit measurements in seconds.

• Task 3 evidence records 7 planned/applied moves, a collision-safe destination, 9 verified CSV records, a zero-move repeat run, and an output-safety rejection.

• Task 4 evidence contains real terminal transcripts for multi-intent routing, clarification recovery, reset, unsupported input, EOF, and Ctrl+C.

Scope and limitations

All Task 3 data is synthetic. Benchmark timings vary by environment. The report does not include Task 1, personal identity, credentials, a certificate decision, track-eligibility confirmation, or a Google Form submission claim.

Table of contents

Section	Contents
1	Task 2 — Fibonacci
2	Task 3 — File automation
3	Task 4 — Rule-based chatbot
4	Cross-task quality assurance
5	Conclusion and appendix

1. Task 2 — Fibonacci generation

Objective and requirements mapping

The module returns exactly the requested number of Fibonacci terms beginning with 0 and 1. It validates negative and invalid-type inputs, emits structured JSON arrays through the CLI, and measures generation with Python's timeit framework.

Requirement	Implemented behavior
Exact list output	Iterative function returns a Python list; CLI emits JSON.
Validation	Negative integers raise ValueError; bool and non-integers are rejected.
Benchmark	timeit measures fibonacci(count) only, excluding printing and JSON encoding.

Algorithm and validation excerpt

```
def fibonacci(count: int) -> list[int]:
    """Return exactly ``count`` Fibonacci terms beginning with 0 and 1.

    The algorithm is iterative. It uses O(count) list storage, and each addition uses
    Python arbitrary-precision integers; additions become more expensive as values
    grow, so the usual constant-cost arithmetic simplification does not apply.
    """
    _validate_count(count)
    terms: list[int] = []
    previous, current = 0, 1
    for _ in range(count):
        terms.append(previous)
        previous, current = current, previous + current
    return terms


def _validate_positive(value: int, *, label: str) -> None:
    if isinstance(value, bool) or not isinstance(value, int):
        raise TypeError(f"{label} must be an integer, not {type(value).__name__}")
    if value <= 0:
```

The loop performs O(count) iterations and stores O(count) integers. Python's arbitrary-precision integers make additions progressively more expensive as the sequence grows, so large-count cost is not constant per iteration.

1. Task 2 — executed outputs and timeit

Fresh command-line evidence

Command case	Observed result
count 10	`[0,1,1,2,3,5,8,13,21,34]`
count 0	`[]`
count 100	Exit 0; retained JSON contains 100 terms and ends with 218922995834555169026.
count -1	Exit 2; `task2.py: error: count must be greater than or equal to zero`

Benchmark methodology and results

timeit: fibonacci(count) only; excludes printing, JSON encoding, and metadata. Units: seconds. Results are measurements from this environment, not performance guarantees.

Terms	Sample totals, seconds (5 repeats × 100)	Min/call, seconds	Median/call, seconds
10	6.6701e-05, 6.512e-05, 6.0401e-05, 6.3222e-05, 6.1196e-05	6.0401e-07	6.3222e-07
100	0.000532159, 0.000664326, 0.000481606, 0.00156766, 0.0022289	4.81606e-06	6.64326e-06
1000	0.0105699, 0.00737288, 0.00741217, 0.0073413, 0.018092	7.3413e-05	7.41217e-05

1. Task 2 — quality evidence

Testing and limitations

The full test suite includes exact small outputs, recurrence behavior, a larger known term, negative and invalid types, JSON CLI behavior, resource limits, benchmark validation, and real timing metadata. The consolidated run passed; the report does not treat timing values as hardware-independent targets.

Reproduction commands

```
python -m internship.task2 --count 10
python -m internship.task2 --count 0
python -m internship.task2 --count -1
python -m internship.task2 --benchmark --benchmark-counts 10 100 1000 --repeats 5 --number 100
```

Evidence reference: evidence/phase5/outputs/task2-count-10.json, task2-count-0.json, task2-count-100.json, task2-invalid.stderr, and task2-benchmark.json.

2. Task 3 — safe file automation

Objective and workflow

Task 3 accepts an explicit synthetic workspace, plans immediate-file organization by final extension, optionally applies collision-safe moves, extracts emails and transaction IDs from relevant UTF-8 text files, and writes a read-back-verified master CSV.

Preview	Apply	Extract and verify
Validate workspace and plan deterministic moves without writing.	Require --apply, preflight UTF-8 input, refuse overwrites, and record outcomes.	Extract ordered records and atomically write/read back a UTF-8 CSV.

Safety boundaries

• Preview is the default; writes require an isolated child under artifacts/task3.

• Canonical fixtures, the artifacts parent, path traversal, symlinks, arbitrary directories, and unsuitable CSV names are rejected.

• Preflight decoding occurs before apply moves; later filesystem races can still produce truthful partial-failure reporting.

• Formula-leading CSV cells are sanitized for spreadsheet safety while raw synthetic evidence remains traceable.

```
def plan_organization(root: str | Path, *, exclude_names: set[str] | None = None) ->
  list[PlannedMove]:
    """Plan deterministic moves of immediate regular files without changing the
      filesystem."""
    workspace = validate_workspace(root)
    reserved: set[Path] = set()
    moves: list[PlannedMove] = []
    for source in _safe_immediate_files(workspace, excluded_names=exclude_names):
        destination = _collision_free_destination(workspace, source, reserved)
        reserved.add(destination)
        moves.append(
            PlannedMove(
                source_path=source.relative_to(workspace).as_posix(),
                destination_path=destination.relative_to(workspace).as_posix(),
            )
        )
    return moves


def apply_plan(root: str | Path, moves: Iterable[PlannedMove]) -> dict[str, Any]:
    """Apply planned moves without overwriting; report every completed or failed move."""
    workspace = validate_workspace(root, writable=True)
```

2. Task 3 — executed organization and CSV

Fresh isolated demonstration

A fresh copy of fixtures/task3 was placed under artifacts/task3/phase5-integration. Preview planned 7 moves without a CSV. Apply completed 7 moves with zero failures and produced 9 verified records. The repeat run planned 0 moves and safely regenerated the verified master CSV.

Before/after organization example

Source	Destination	Outcome
.metadata	no_extension/.metadata	completed
archive.tar.GZ	gz/archive.tar.GZ	completed
empty.txt	txt/empty.txt	completed
inbound.LOG	log/inbound.LOG	completed
notes.txt	txt/notes__1.txt	completed
readme	no_extension/readme	completed
report.TXT	txt/report.TXT	completed

The existing txt/notes.txt caused the top-level notes.txt to move to txt/notes__1.txt rather than overwrite content.

Verified master CSV excerpt (synthetic data)

```
source_path,line_number,kind,value
log/inbound.LOG,2,email,demo.alpha@example.test
log/inbound.LOG,2,email,demo.beta@example.test
log/inbound.LOG,2,transaction_id,TXN-A1B2C3
log/inbound.LOG,2,transaction_id,TXN-Z9Y8X7
log/inbound.LOG,3,transaction_id,TXN-A1B2C3
```

2. Task 3 — integrity, idempotency, and errors

Integrity and regression evidence

• The multiset of SHA-256 file-content hashes before and after organization matched after excluding the generated CSV and marker.

• A second apply planned zero moves and retained nine verified records.

• A deliberate --csv notes.txt attempt exited 2 before any move or write; input hashes remained unchanged.

• The full suite exercises preview/apply, collisions, idempotency, symlink/unsafe-root rejection, malformed UTF-8 preflight, CSV read-back, formula protection, and CLI error exits.

Relevant implementation excerpt

```
def write_master_csv(root: str | Path, output: str | Path, records: Iterable[Record]) ->
  list[dict[str, str | int]]:
    """Atomically write and read back a sanitized master CSV inside a workspace."""
    workspace = validate_workspace(root, writable=True)
    target = _validate_csv_target(workspace, output)
    exported = [_export_record(record) for record in records]
    temporary_name: str | None = None
    try:
        try:
            with tempfile.NamedTemporaryFile("w", encoding="utf-8", newline="",
              dir=workspace, prefix=".task3-", suffix=".tmp", delete=False) as
              temporary:
                temporary_name = temporary.name
                writer = csv.DictWriter(temporary, fieldnames=CSV_FIELDS,
                  lineterminator="\n")
                writer.writeheader()
                writer.writerows(exported)
            os.replace(temporary_name, target)
            temporary_name = None
        except OSError as error:
            raise ExtractionError(f"could not safely write {target.name}: {error}")
              from error
    finally:
        if temporary_name:
            Path(temporary_name).unlink(missing_ok=True)
    try:
        with target.open("r", encoding="utf-8", newline="") as csv_file:
            read_back = list(csv.DictReader(csv_file))
    except OSError as error:
        raise ExtractionError(f"could not read back {target.name}: {error}") from error
    expected = [{key: str(row[key]) for key in CSV_FIELDS} for row in exported]
    if read_back != expected:
```

Evidence references: evidence/phase5/outputs/task3-preview.json, task3-apply.json, task3-repeat.json, task3-master.csv, and task3-safe-failure.stderr.

3. Task 4 — multi-intent rule-based chatbot

Architecture and state

The terminal bot is local and rule based. It normalizes Unicode NFKC input, casefolds, replaces punctuation, normalizes whitespace, applies functional dictionary routing, and uses nested if/elif/else logic for pending choices. It has task guidance, Python learning, and submission guidance trees.

State field	Purpose
current_tree / pending_choice	Retains the current help tree and the next valid contextual choices.
last_intent	Records each actual recognized or reported intent, including contextual follow-ups.
turn_count / history	Counts turns and keeps the 20 newest normalized inputs in process only.
clarification_count	Escalates a repeated pending-choice miss; reset/cancel clears it.

Normalization excerpt

```
def normalize(text: str) -> str:
    """Normalize Unicode input with NFKC, casefolding, punctuation and whitespace."""
    if not isinstance(text, str):
        raise TypeError("text must be a string")
    normalized = unicodedata.normalize("NFKC", text).casefold()
    characters = [" " if unicodedata.category(char).startswith("P") else char for char in
      normalized]
    return " ".join("".join(characters).split())


def _tokens(text: str) -> set[str]:
    return set(text.split())
```

3. Task 4 — routing and recovery

Routing/state-transition flow

Input
↓ normalize: NFKC, casefold, punctuation, whitespace
↓ global priority: exit → reset/cancel → help
↓ explicit topic switches: tasks / Python / submission
↓ pending contextual tree: nested choices
↓ first ambiguity: contextual clarification
↓ repeated ambiguity: explicit choices + help/reset
↓ unknown: helpful fallback

The implementation gives explicit topic switches priority over earlier pending state. Recognized input resets clarification recovery; reset/cancel clears context, history, fallback count, and clarification count.

Routing excerpt

```
        def handle_turn(text: str, session: SessionState) -> TurnResult:
            """Route one user turn and mutate only the supplied in-memory session."""
            normalized = normalize(text)
            _record(session, normalized)
            intent = detect_intent(normalized, session)
            if intent == "exit":
                session.active = False
                session.last_intent = "exit"
                return TurnResult("Goodbye. This session was local and no information was saved
                  or submitted.", "exit", True)
            if intent == "reset":
                session.current_tree = None
                session.pending_choice = "root_choice"
                session.last_intent = "reset"
                session.history = [normalized]
                session.fallback_count = 0
                session.clarification_count = 0
                return TurnResult("Context reset. Choose tasks, python, or submission.",
                  "reset")
            if intent == "blank":
                session.last_intent = "blank"
                return TurnResult("Please enter a topic, or type help for tasks, python, and
                  submission guidance.", "blank")
            if intent in INTENT_HANDLERS:
                return INTENT_HANDLERS[intent](session, normalized)
            if intent == "pending":
                result = _pending_response(session, normalized)
                session.last_intent = result.intent
                if result.intent != "ambiguous":
                    session.clarification_count = 0
                    session.fallback_count = 0
                return result
            session.fallback_count += 1
            session.last_intent = "unknown"
            if session.fallback_count >= 2:
                return TurnResult("I still do not recognize that request. Type help, or reset to
                  clear the current context.", "unknown")
            return TurnResult("I do not recognize that request. Try tasks, python, submission, help,
              reset, or exit.", "unknown")
```

3. Task 4 — genuine conversations and limits

Executed multi-intent transcript

```
Welcome to the Progree Python assignment helper. Choose: tasks, python, or submission. Type help
  at any time; reset/cancel clears context; exit ends this local session.
you> I can help with internship tasks (2, 3, 4), Python learning, or submission guidance. Which
  area would you like?
you> Task 2 generates exactly count Fibonacci terms, validates invalid input, emits JSON, and
  benchmarks with timeit. Reproduce it with: python -m internship.task2 --count 10. Ask about
  output, validation, or benchmarking.
you> Benchmark Task 2 with: python -m internship.task2 --benchmark --benchmark-counts 10 100
  1000 --repeats 5 --number 100.
you> Python learning: choose fundamentals, lists and loops, functions and validation, or regex
  and file handling.
you> That is ambiguous here. Choose fundamentals, lists and loops, functions and validation, or
  regex and file handling.
you> I still need a clear choice. Choose fundamentals, lists and loops, functions and
  validation, or regex and file handling. Type help for the main menu or reset to clear this
  context.
you> Lists keep ordered values. A for loop visits each item: for value in values: ... . For Task
  2, the list stores Fibonacci terms in generation order. Ask about functions, regex, or another
  Python topic.
you> Submission guidance: the notice says complete at least 3 of 4 tasks and combine work into
  one PDF. It names 18 October 2026, but no exact deadline hour is verified. Ask about the PDF,
  deadline, or Task 1.
you> The supplied notice names 18 October 2026. Its exact cutoff time and timezone are not
  confirmed.
you> I can clarify the three-of-four rule, one-PDF requirement, deadline date, or optional Task
  1. Which one?
you> Context reset. Choose tasks, python, or submission.
you> Goodbye. This session was local and no information was saved or submitted.
```

Unsupported Task 4 input and recovery

```
you> Task 4 is this local rule-based terminal chatbot. It normalizes text, routes intents with a
  dictionary, tracks session state, and handles fallbacks. It is being used now; final PDF work
  is not part of this session.
you> I do not recognize that as a Task 4 question. Ask about operation, intents, normalization,
  or session state; or type help/reset.
you> I still need a clear choice. Choose operation, intents, normalization, or session state.
  Type help for the main menu or reset to clear this context.
you> The session keeps the current tree, pending choice, last intent, turn count, and only the
  most recent 20 normalized turns in memory. It is not persisted.
you> Goodbye. This session was local and no information was saved or submitted.
```

Separate executed captures confirm EOF ends with “End of input” and SIGINT ends with “Interrupted safely”; both exit 0. No session state is persisted and no external service, credentials, or form submission is used.

4. Cross-task quality assurance

Consolidated verification

Check	Fresh observed result
Automated tests	54 tests passed in 1.678 seconds.
Task 2 timeit	Three counts, five repeat totals each, 100 executions per repeat; units are seconds.
Task 3 integrity	Content-hash multiset matched; safe failure preserved inputs; repeat planned zero moves.
Task 4 termination	Scripted exit, EOF, and Ctrl+C all captured with exit 0.
Original documents	All four original assignment documents matched the approved-starting-commit SHA-256 digests.

Environment and reproducibility

Python 3.12.14; platform: Linux-6.18.44-x86_64-with-glibc2.41; x86_64. Fresh source commit: `0ead4fcfc3a94d928af7e7827793a06c7fc17352`. Consolidated manifest: evidence/phase5/manifest.json. It records exact commands, exit codes, UTC execution timestamp, retained-output paths, fixture references, and SHA-256 values.

5. Conclusion

Tasks 2–4 are implemented and freshly verified in this repository: iterative Fibonacci generation with parameter checks and timeit evidence; safe synthetic-workspace organization and text parsing with verified CSV output; and a stateful rule-based terminal chatbot with help trees, recovery, and graceful termination.

Skills demonstrated

• Python API and CLI design, validation, JSON output, and timing methodology.

• Filesystem safety boundaries, deterministic organization, regular expressions, CSV read-back, and integrity checks.

• Unicode normalization, explicit state machines, dictionary dispatch, nested contextual decisions, and terminal resilience.

• Evidence provenance, report automation, and independent PDF quality assurance.

Outstanding human verification

• Python Programming track eligibility and organizer acceptance of Tasks 2–4.

• Certificate name, registration ID, submission email/WhatsApp number, and form upload constraints/accessibility.

• Organizer deadline cutoff hour and timezone; only the date 18 October 2026 is supplied.

• Authorized Google Form submission after independent review.

Future improvements could expand the chatbot vocabulary, add configurable Task 3 patterns, and compare benchmark trends across controlled environments. They are not claimed as completed here.

Appendix. Reproduction commands and references

Exact command forms recorded in the Phase 5 manifest

```
python -m unittest discover -s tests -v
python -m internship.task2 --count 10
python -m internship.task2 --count 0
python -m internship.task2 --count 100
python -m internship.task2 --count -1
python -m internship.task2 --benchmark --benchmark-counts 10 100 1000 --repeats 5 --number 100
python -m internship.task3 --root artifacts/task3/phase5-integration
python -m internship.task3 --root artifacts/task3/phase5-integration --apply --csv master.csv
python -m internship.task3 --root artifacts/task3/phase5-integration --apply --csv master.csv
python -m internship.task3 --root artifacts/task3/phase5-safe-failure --apply --csv notes.txt
```

Repository references

Purpose	Path
Authoritative assignment	Python Programming Tasks & Instructions.pdf
Submission messages	Text Message 1.md; Text Message 2.md
Phase 5 manifest	evidence/phase5/manifest.json
Report generator	tools/generate_report.py
Task sources	internship/task2.py; internship/task3.py; internship/task4.py

The report's displayed execution outcomes come from retained Phase 5 evidence files validated by their SHA-256 digests before PDF generation.